

Contents

guide 01 the pile	1
0. 先给新手的结论	1
1. 回到当时的语境	2
2. 论文主张拆成问题、方法、证据	2
3. The Pile 到底是什么	3
4. 数据混合的系统图	4
5. 权重与 epoch 的设计理由	4
6. Pile-CC: 为什么不用现成 WET	5
7. OpenWebText2、GitHub、ArXiv 等子集的设计直觉	5
8. 训练集也可以是 benchmark	6
9. GPT-2 / GPT-3 在 Pile 上暴露了什么	7
10. 核心实验: Pile vs CC-100 vs Raw CC	7
11. 为什么数据过滤不能只追求“干净”	8
12. 去重、污染和 13-gram	8
13. 结构统计与数据文档化	9
14. 论文中的几个技术细节为什么重要	9
15. 和本仓库代码怎么连起来	10
16. 代码样例: 把 Pile 的权重思想写成玩具采样器	10
17. 代码样例: BPB 计算	11
18. 代码样例: 13-gram contamination 检测	11
19. 对现在的意义	12
20. 局限和批判性阅读	12
21. 用 AI agent 正确学习这篇论文	12
22. 阅读后的闭卷问题	13
23. 一页复盘	14
24. 本仓库下一步学习动作	14

guide_01_the_pile

原论文: The Pile: An 800GB Dataset of Diverse Text for Language Modeling

本地原文 PDF: [learning/data-curation/paper/01_the_pile.pdf](#)

作者: Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, Connor Leahy, EleutherAI

年份: 2020 / arXiv 2021

读法定位: 这不是一篇“模型结构论文”，而是一篇把大模型数据工程公开化、可复现化、可争论化的数据集系统论文。读懂它，你会开始把“训练数据”看成一个有配方、权重、抽样、去重、污染、评测和伦理边界的工程对象。

0. 先给新手的结论

The Pile 的主张很直接：只靠 Common Crawl 这样的网页抓取不够，通用语言模型需要覆盖更广的知识分布。因此作者构建了一个 825.18 GiB 的英文训练语料，把 22 个来源混在一起，其中既有网页，也有论文、医学、代码、法律、问答、书籍、字幕、专利、邮件、IRC 等材料。它的贡献不是“一个神奇清洗规则”，而是把开放 LLM 预训练的数据配方第一次做得足够大、足够透明，并且用实验说明：在同等 40GB

训练预算下，用 Pile 训练的 1.3B 模型比 Raw Common Crawl 和 CC-100 在 Pile 各子域上都更好，同时传统 WikiText / LAMBADA 指标也没有明显牺牲。

这篇论文真正教你的有四件事：

1. 数据不是越大越好，而是“覆盖哪些分布、如何给权重、如何过滤、如何留出评测集”的组合问题。
2. 训练集也可以变成 benchmark。Pile 的 22 个子集可以暴露模型在哪些领域不擅长。
3. 数据质量过滤不能只追求“像 Wikipedia”，否则会把代码、数学、论坛、论文、法律这种真正有价值的分布过滤掉。
4. 大规模数据集必须被文档化。来源、处理、同意、版权、毒性、偏见、污染风险，都应该进入技术报告，而不是藏在“we train on web data”一句话里。

如果你只记一句话：The Pile 把“LLM 数据配方”从黑盒经验变成了可检查、可复现、可批评的工程系统。

1. 回到当时的语境

2020 年前后，大模型路线已经很清楚：Transformer 结构稳定，扩大参数量和训练 token 会持续带来能力提升。GPT-2、T5、Megatron-LM、GPT-3 都说明了这件事。可是另一个问题开始变得尖锐：参数可以扩，算力可以堆，数据从哪里来？

当时最常见的答案是 Common Crawl。Common Crawl 覆盖巨大，但它本质是网页抓取，问题也很明显：

- 原始网页包含导航栏、广告、页脚、错误页、脚本残留、模板文字。
- 纯网页分布偏向大众互联网，对论文、代码、法律、医学、数学、专业问答覆盖不足。
- 强过滤如果以 Wikipedia 或高质量网页为正例，会把“不像网页百科”的高价值数据也删掉。
- 很多论文只说用了 large web corpus，没有充分说明来源、处理流程、重复、污染和版权问题。

The Pile 的出现，就是为了回答一个非常基础但当时不够公开的问题：如果我们想训练开放的大语言模型，能不能给出一个可下载、可复现、来源透明、覆盖多领域的训练集？

注意这里的关键不是“Pile 比所有数据都干净”。恰好相反，作者承认它包含争议、噪声、偏见和版权/同意问题。它的技术贡献在于把这些问题摆到台面上，并提供处理代码和分析，让后续研究者能基于明确的数据对象做决策。

2. 论文主张拆成问题、方法、证据

问题：语言模型越来越大，训练数据需求越来越高；只依赖 Common Crawl 会限制跨领域泛化，而且数据来源不透明会让复现、审计、污染分析和伦理讨论都变困难。

方法：构建 22 个子集组成的 825.18 GiB 英文语料。不同子集按质量和大小设置 epoch 权重；高质量、较小、专业性强的数据会被上采样。作者还提供 train / validation / test 划分、处理代码、数据文档、结构统计和风险分析。

证据：作者训练了结构相同的 1.3B 模型，分别使用 Pile、CC-100 英文部分和 Raw Common Crawl。为了公平，每个训练集都去除与评测集重叠的 13-gram，并下采样到约 40GB。结果是 Pile 模型在 Pile 的所有子集上都优于两个 Common Crawl 基线，在 WikiText 上也更好，在 LAMBADA 上基本持平。

局限：没有做全 Pile 范围的去重；主要是英文；含有版权、同意、偏见、毒性和污染风险；质量过滤标准今天看已经偏早期；很多结论来自 1.3B 和 40GB 控制实验，不能直接推出所有规模下的最优配方。

这条链条要一直抓住：数据混合多样性 -> 覆盖更多分布 -> 模型跨领域 BPB 更低 -> 下游传统指标不明显受损 -> 但数据治理成本上升。

3. The Pile 到底是什么

The Pile 是一个英文文本训练语料，原始总量 825.18 GiB。因为部分子集会被上采样，一个完整 epoch 的有效大小约 1254.20 GiB。它由 22 个子集构成，作者把每个子集的 Raw Size、Weight、Epochs、Effective Size 和平均文档大小列成表。

用学习者视角，可以把 22 个子集分成六类：

网页与开放互联网

- Pile-CC: 227.12 GiB，权重 18.11%，1 epoch。它是作者自己从 Common Crawl WARC 里抽取和过滤的高质量英文网页。
- OpenWebText2: 62.77 GiB，权重 10.01%，2 epochs。它沿着 WebText/OpenWebText 思路，用 Reddit 外链投票作为网页质量代理。
- HackerNews: 3.90 GiB，权重 0.62%，2 epochs。它提供技术社区讨论风格。
- YouTube Subtitles: 3.73 GiB，权重 0.60%，2 epochs。它提供口语、教育、流行文化等字幕文本。

学术与专业文本

- PubMed Central: 90.27 GiB，权重 14.40%，2 epochs。医学/生物医学全文论文。
- ArXiv: 56.21 GiB，权重 8.96%，2 epochs。学术论文 LaTeX 源码转换后的文本。
- PubMed Abstracts: 19.26 GiB，权重 3.07%，2 epochs。医学论文标题和摘要。
- PhilPapers: 2.38 GiB，权重 0.38%，2 epochs。哲学论文。
- NIH ExPorter: 1.89 GiB，权重 0.30%，2 epochs。NIH 资助项目摘要。

书籍与长上下文

- Books3: 100.96 GiB，权重 12.07%，1.5 epochs。
- Gutenberg / PG-19: 10.88 GiB，权重 2.17%，2.5 epochs。
- BookCorpus2: 6.30 GiB，权重 0.75%，1.5 epochs。

这些数据提供长文结构、叙事连贯性和更长距离的依赖关系。对后来长上下文训练来说，书籍数据很重要。

代码、数学、问答

- GitHub: 95.16 GiB，权重 7.59%，1 epoch。作者实际收集了更多 GitHub 数据，但随机采样约 95 GiB 进入 Pile。
- Stack Exchange: 32.20 GiB，权重 5.13%，2 epochs。问答格式，覆盖技术、科学、生活等多主题。
- DM Mathematics: 7.75 GiB，权重 1.24%，2 epochs。DeepMind 数学数据，提供符号推理和数学题。

法律、专利、政治机构

- FreeLaw: 51.15 GiB，权重 6.12%，1.5 epochs。美国法院意见。
- USPTO Backgrounds: 22.90 GiB，权重 3.65%，2 epochs。专利背景部分。
- EuroParl: 4.59 GiB，权重 0.73%，2 epochs。欧洲议会文本。

对话、邮件、社区记录

- OpenSubtitles: 12.98 GiB，权重 1.55%，1.5 epochs。
- Ubuntu IRC: 5.52 GiB，权重 0.88%，2 epochs。
- Enron Emails: 0.88 GiB，权重 0.14%，2 epochs。

这个列表本身就是论文的核心：作者不是简单说“我们用了很多网页”，而是在告诉你模型会吃到哪些社会、专业和表达风格分布。

4. 数据混合的系统图

把 The Pile 当成一个数据系统，它的流程可以画成这样：

```
source datasets
|
|   source-specific extraction
|   - WARC -> justext text
|   - JATS / TeX / XML / HTML / email -> text
|   - code repos -> text files
v
component json/text documents
|
|   filtering and cleanup
|   - language filter
|   - quality classifier / heuristics
|   - source-specific boilerplate removal
v
component-level corpora
|
|   selected dedup
|   - MinHashLSH for OWT2 and Pile-CC
|   - no full Pile-wide dedup
v
weighted mixture sampler
|
|   weight roughly = document_count * desired_epochs
v
30 output piles / train stream
|
|   holdout and split
v
train + validation + test
```

这里有两个很重要的工程判断。

第一，Pile 不是统一清洗规则。每个来源有自己的抽取器和处理方式。论文附录 C 大量篇幅就是在讲每个子集怎么拿、怎么抽、怎么过滤。数据工程里最危险的幻想之一，就是以为一个 universal filter 能处理所有文本。

第二，Pile 没有做全局去重。作者明确说，由于内存限制，没有进行 Pile-wide deduplication，只在最容易重复的 OpenWebText2 和 Pile-CC 内部做文档级去重。这一点今天看是重要局限：跨源重复、benchmark 污染、公共网页镜像，仍可能留在数据里。

5. 权重与 epoch 的设计理由

The Pile 不是把 22 个子集简单拼接。作者借鉴 GPT-3 的做法，对高质量或较小的数据上采样。比如 Wikipedia 体积只有 6.38 GiB，但被看 3 次；PubMed Central、ArXiv、Stack Exchange、DM Mathematics 等专业子集大多是 2 epochs。

抽象成一个采样分布，设第 (i) 个子集有 (n_i) 个文档，目标 epoch 数是 (e_i)，则采样时可以理解为：

$$q_i = \frac{n_i e_i}{\sum_j n_j e_j}$$

真实实现还会受到文档大小影响，所以最终论文表里用 bytes 展示 Weight 和 Effective Size。学习时不用纠结这一点，关键是明白：epoch 是一种把“作者认为更重要的分布”显式注入训练流的方式。

这个设计背后的判断是：

- 学术、医学、法律、数学、代码等专业材料质量高，但相对网页小，如果按原始字节比例混合，会被网页淹没。
- 上采样不能无限高，否则模型会记忆小数据集，重复 exposure 还可能放大偏见和污染。
- 作者设置了上限：不让任何数据超过 3 epochs，并尽量避免超过 2 epochs。

从今天的视角看，这就是早期 data mixture engineering。后来的 DoReMi、DCLM、FineWeb、Llama 3 数据配比，都在更系统地回答同一个问题：不同数据域应该以什么比例进入训练？

6. Pile-CC: 为什么不用现成 WET

论文里很值得细读的是 Pile-CC，因为它解释了为什么 Common Crawl 很难用。

Common Crawl 主要有两种格式：

- WARC: 原始网页响应，包含 HTML 和元数据。
- WET: Common Crawl 预抽取的纯文本。

WET 的优点是省算力、省带宽；缺点是抽取质量差，经常保留菜单、页脚、模板和大量 boilerplate。作者认为只做文档级过滤不足以修复 WET，因为很多噪声在文档内部。也就是说，一篇网页既可能有正文，也可能混着导航栏和模板文字，整篇删掉太浪费，整篇保留又太脏。

所以 Pile-CC 选择从 WARC 开始，用 jusText 做正文抽取。作者也考虑过 trafilatura、Newspaper、Goose3、DragNet。最后选 jusText 的理由很实用：它倾向于丢掉更多内容，但在海量 Common Crawl 场景下，宁可少要一点，也要降低 boilerplate。

Pile-CC 的过滤也有一个关键选择：用 OpenWebText2 作为高质量正例训练 fastText 分类器，而不是用整个 Pile 做正例。作者实验发现，用整个 Pile 做正例会让分数分布敏感性较低。最后他们使用类似 GPT-3 的 Pareto thresholding，并在附录表中展示不同 alpha 下的保留比例；他们选择 alpha = 3，保留比例约 0.2390。

这给本仓库学习的启发是：网页数据清洗不是一句“filter bad docs”就完了。至少要分成：

```
raw web response
-> language detection
-> main-content extraction
-> quality scoring
-> deduplication
-> metadata retention
```

本仓库的 learning/data-curation/src/cc_extract.py 用 trafilatura 做教学版抽取，正好对应论文里的 WARC/WET 与正文抽取问题。虽然实现选择不同，但学习目标一致：不要把 HTML 噪声直接喂给模型。

7. OpenWebText2、GitHub、ArXiv 等子集的设计直觉

The Pile 的 22 个子集不是随机堆材料。每个来源都承担一种“能力补丁”。

OpenWebText2 用 Reddit 外链投票作为质量代理，这是 WebText 思路的开放复刻。直觉是：被人主动分享和投票的网页，比随机网页更可能含有可读正文。它还做了 URL 去重和文档级 MinHash 去重。

ArXiv 用 TeX 源码经 pandoc 转 Markdown。它的意义不是让模型背论文，而是引入长篇学术写作、公式、引用、术语和技术论证结构。论文也指出，如果希望模型能生成论文或理解科研文本，ArXiv 是重要分布。

GitHub 的构造使用 stars 作为质量代理，只收集超过 100 stars 且小于 1GB 的仓库，然后抽取文本文件。作者实际收集了 630.64 GiB GitHub 数据，最后随机采样 95.0 GiB 进入 Pile。这里的关键是：代码不是自然语言，但语言模型越来越需要代码能力和符号结构。

Stack Exchange 提供问答结构。作者保留问题和高赞回答，并用 Q: / A: 形式提供上下文。这其实是在预训练阶段就给模型一些“问题 -> 解答”的弱结构，虽然它还不是 instruction tuning。

PubMed、FreeLaw、USPTO、PhilPapers、NIH 等专业文本的意义，是让模型接触普通网页很少覆盖的概念密度、术语体系和论证文体。论文后面的实验也说明，GPT-3 在这类研究/专业文本上相对弱，说明这些分布不是 web/books 训练自然就能完全覆盖的。

8. 训练集也可以是 benchmark

The Pile 不只是训练集，也被作者当成跨领域评测集。原因很简单：如果一个数据集覆盖 22 个明显不同的领域，那模型在每个子集上的 loss 就能告诉你它对哪些分布不熟。

论文提供 train / validation / test 切分。validation 和 test 各占 0.1%。这个比例听起来小，但因为总量巨大，每个 split 都超过 1 GiB。

作者推荐的指标是 BPB，也就是 bits per UTF-8 encoded byte。它比 perplexity 更适合跨数据域、跨 tokenizer 比较。公式是：

$$\text{BPB} = \frac{L_T}{L_B} \log_2(e^\ell) = \frac{L_T}{L_B} \frac{\ell}{\ln 2}$$

其中：

- (ℓ) : 语言模型输出的平均 negative log likelihood。
- (L_T) : token 数。
- (L_B) : UTF-8 字节数。
- (L_T / L_B) : token-per-byte 比例，和 tokenizer 有关。

为什么不用普通 perplexity？因为不同 tokenizer 会把同一段文本切成不同 token 数。代码、数学、Unicode、多语言片段尤其明显。BPB 把 loss 归一到 byte，能减少 tokenizer 切分差异带来的混淆。

可以把评测张量流画成这样：

```
document text
-> UTF-8 bytes length L_B
-> tokenizer
-> token ids [T]
-> segment into [S <= context_length]
-> model predicts next-token logits [S, vocab]
-> NLL loss ell
-> BPB = (T / L_B) * ell / ln(2)
```

本仓库后续做 tokenizer 或数据清洗实验时，BPB 这个思想很重要：如果只看 token-level perplexity，可能把 tokenizer 的压缩差异误读成模型能力差异。

9. GPT-2 / GPT-3 在 Pile 上暴露了什么

论文先用 GPT-2 和 GPT-3 做 zero-shot Pile test。它们没有在 Pile 上 fine-tune。结果整体符合 scaling law：模型越大，BPB 越低。作者在 GPT-3 系列上拟合了一条关系，线性拟合系数约为 -0.1674，截距约 2.5516。

更有意思的是 component-wise 分析。直接比较不同子集的 BPB 不公平，因为子集本身熵不同，数学和代码天然更难，网页和书籍可能更接近 GPT 训练分布。因此作者构造了一个相对指标：

$$\Delta_s = (L_s^{GPT3} - L_{OWT2}^{GPT3}) - (L_s^{GPT2Pile} - L_{OWT2}^{GPT2Pile})$$

直觉解释：

- 用 OpenWebText2 当参照，因为它接近 GPT-3 训练里的网页分布。
- GPT2-Pile 在 Pile 上训练过，所以它对各子集的差异更像“子集本身难度”。
- GPT-3 和 GPT2-Pile 的相对差，可以作为 GPT-3 缺少某些 Pile 分布的代理信号。

结果显示，GPT-3 在研究/学术文本如 PubMed Central、PubMed Abstracts、ArXiv，专业域如 FreeLaw、HackerNews、USPTO，以及非自然语言形态如 GitHub、DM Mathematics 上相对弱。这个结论很重要：大模型可以泛化，但训练分布缺失仍会留下明显能力空洞。

10. 核心实验: Pile vs CC-100 vs Raw CC

为了证明 Pile 作为训练集真的有价值，作者训练了三个结构相同的 1.3B 模型：

- The Pile。
- CC-100 英文部分。
- Raw Common Crawl WET 英文样本。

为了公平：

- 三个训练集都下采样到约 40GB。
- 都用与 GPT-3 类似的 13-gram overlap filter 去除与评测集重叠的片段。
- 模型结构一致。
- 评测包括 Pile validation/test、WikiText、LAMBADA。

主结果可以压缩成下面这张表：

Training data	Pile val BPB	Pile test BPB	WikiText PPL	LAMBADA PPL	LAMBADA ACC
The Pile	0.9281	0.9433	5.59	12.78	50.1
CC-100	1.3143	1.3293	8.27	11.78	49.7
(en)					
Raw CC	1.1180	1.1275	11.75	19.84	43.8

这张表要这样读：

- Pile 在 Pile val/test 上明显更低，说明它覆盖 Pile 的 22 个域更好。
- Pile 在 WikiText 上也明显更好，说明多域混合没有牺牲传统语言建模能力。
- LAMBADA 上 Pile 的 perplexity 比 CC-100 略差，但 accuracy 基本相当；所以作者说变化可以忽略。

- Raw CC 在 Pile BPB 上比 CC-100 好，但在 LAMBADA/WikiText 上差。这引出一个重要解释：CC-100 的 perplexity-based filtering 以 Wikipedia 模型为核心，会丢弃太像或太不像 Wikipedia 的文本，从而限制多样性。

真正的证据链在 Table 4。Pile 训练模型在每一个 Pile 子集上都是最优，包括 ArXiv、GitHub、DM Mathematics、EuroParl、PubMed、FreeLaw 等。也就是说，Pile 的收益不是来自某一个漂亮 benchmark，而是来自跨域 loss 的系统性降低。

11. 为什么数据过滤不能只追求“干净”

The Pile 论文对今天的数据清洗仍然有启发：过度追求“像高质量百科文本”会让模型失去多样性。

CC-100 的过滤方式大致是训练一个 Wikipedia 风格的语言模型，然后丢掉 perplexity 太高或太低的数据。这个策略能去掉很多垃圾，但也会误删：

- 代码，因为 token 分布不像自然语言。
- 数学题，因为符号、变量和格式让 perplexity 异常。
- 论坛和问答，因为口语、片段化、引用结构与百科不同。
- 法律/专利/论文，因为长句、术语和格式特殊。
- 多语言和非标准英语片段。

所以清洗不是简单地把 distribution tail 切掉。你要先问：这个 tail 是垃圾，还是模型未来需要学会的专业分布？

这也是本仓库 `quality_filter.py` 要配合 `data_mix_ablation.py` 一起看的原因。质量过滤和数据配比必须共同设计。一个过滤器让所有文本都“像教科书”，短期 loss 可能好看，长期能力谱会变窄。

12. 去重、污染和 13-gram

论文在去重上很诚实：没有做全 Pile 范围去重，只在 OpenWebText2 和 Pile-CC 内部做 MinHashLSH 文档级去重。参数上，作者使用 Python Datasketch 库，每个 MinHash 用 10 个 hash functions，近似 Jaccard similarity 约 0.5。结果 OpenWebText2 重复率约 28%，Common Crawl 约 26%。

MinHash 的核心思想是：如果两个文档的 shingle 集合很像，它们的 MinHash 签名也会以相近概率相同。设两个集合的 Jaccard 相似度为：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

MinHash 近似保留这个相似度；LSH 再把相似签名放入相同 bucket，避免两两比较所有文档。否则 ($O(n^2)$) 比较在几千万文档上完全不可行。论文说朴素比较可能需要数十万年，这不是夸张修辞，而是大规模去重的真实量级。

污染方面，作者在训练对比实验里使用 13-gram overlap filtering 来去除与评测集重叠的实例。这和 GPT-3 论文中的做法一致。13-gram 也被用于论文的探索性分析：作者统计 Common Crawl 中所有 13-gram，发现大量高频片段来自样式字符、HTML 转义、论坛模板、登录提示、错误信息等。

这说明两个问题：

- 重复不只是整篇文章重复，常见 boilerplate 也会在局部片段层面大量重复。
- benchmark contamination 不可能靠“感觉数据很大所以不影响”解决，必须显式查重。

本仓库的 `learning/data-curation/src/minhash_dedup.py` 是这个思想的教学版：它用 char 5-gram 生成 MinHash 签名，再用 LSH 找近邻。和论文参数不完全一致，但机制对齐。


```
def shingles(text: str, n: int = 5):
    text = text.lower().replace("\n", " ")
    for i in range(len(text) - n + 1):
        yield text[i:i + n]

def dedup(docs, threshold=0.7, num_perm=128):
    lsh = MinHashLSH(threshold=threshold, num_perm=num_perm)
    for doc_id, text in docs:
        sig = minhash_sig(text, num_perm=num_perm)
        neighbors = lsh.query(sig)
        if neighbors:
            mark_as_duplicate(doc_id, neighbors[0])
        else:
            lsh.insert(doc_id, sig)
```

学习时要特别注意：去重阈值不是越高越好，也不是越低越好。阈值高会漏掉改写重复，阈值低会误删相似主题但不同内容的文档。数据清洗永远是 precision/recall 的工程权衡。

13. 结构统计与数据文档化

论文第 5、6 节把 The Pile 当成一个需要审计的数据产品来分析。

结构统计包括：

- 文档长度分布：大多数文档很短，但存在长尾超长文档。
- GPT-2 tokenizer 的 bytes-per-token：它粗略反映每个子集相对 WebText 的语法和符号差异。
- 语言比例：用 fastText 估计，Pile 约 97.4% 是英语。作者也提醒，低资源语言识别不可靠，不能据此做精细多语言结论。

主题分析包括：

- 对每个组件的验证集训练 16-topic LDA。
- 用 Pile-CC 的 topic model 去评估其他组件。
- 高 perplexity 表示主题上更偏离 Pile-CC。

结果显示，GitHub、PhilPapers、EuroParl 等明显偏离 Pile-CC。这支持作者的核心主张：Pile 的许多组件不是普通网页的冗余副本，它们确实提供不同主题和表达模式。

风险文档化包括：

- Pejorative content: 使用 profanity-checker 估计各子集的冒犯性词汇比例。
- Bias and sentiment co-occurrence: 分析性别、宗教、种族相关词的共现和情感倾向。
- Author consent and public data: 按 Public、ToS、Author consent 三个维度梳理各子集。
- Broader impacts: 讨论版权、AI timeline acceleration、负面输出等问题。

这部分不是“论文附加道德声明”，而是数据集论文的核心。因为数据决定模型会吸收什么社会分布。一个开放数据集如果只报告 loss，不报告来源、同意和风险，就不完整。

14. 论文中的几个技术细节为什么重要

BPB 的设计理由

如果两个模型用不同 tokenizer，普通 perplexity 不能直接比。BPB 归一到 UTF-8 byte，可以把模型 loss 和 tokenizer token 数差异拆开。对代码、数学、多语言、Unicode 文本尤其重要。

13-gram contamination filtering

13-gram 是一种低成本精确重叠检测。它不能发现语义改写污染，但能发现大量直接拷贝。对大规模训练集来说，这是最基本的防线。

不做全局去重的后果

作者受限内存和工程条件，只做了部分去重。这意味着 Pile 的 benchmark split 与 train split、不同组件之间、以及未来下游 benchmark 都可能存在残留重叠。后来的数据集工程更重视全局去重和 benchmark decontamination。

用上采样表达价值判断

数据权重不是客观真理，而是研究者对“哪些分布值得模型多看”的判断。The Pile 把这个判断写进表格，这是进步；但它仍是人工经验，不是自动最优。

数据文档化本身是技术

Datasheets、data statements、consent table、bias co-occurrence，这些不是旁枝。它们是让数据集可以被复用、被审计、被批评的技术基础。

15. 和本仓库代码怎么连起来

本专题的代码不是要复刻 825 GiB Pile，而是把论文的关键机制缩小成能在本机跑的小实验。

核心对应关系：

- learning/data-curation/src/cc_extract.py: 对应 Pile-CC 的网页抽取问题。论文用 jusText，本仓库教学版用 trafilatura，重点都是从 HTML/WARC 中抽正文并保留 metadata。
- learning/data-curation/src/minhash_dedup.py: 对应 OpenWebText2/Pile-CC 的 Min-HashLSH 去重。
- learning/data-curation/src/simhash_dedup.py: 补充另一类近重复检测思路。
- learning/data-curation/src/quality_filter.py: 对应 C4/Gopher/FineWeb 风格质量过滤，帮助你理解“干净”和“多样”之间的冲突。
- learning/data-curation/src/data_mix_ablation.py: 对应 Pile 的 mixture weights / epochs 思想，用 toy perplexity 看配比变化。
- learning/data-curation/src/capstone_mini_corpus.py: 把 extract -> dedup -> quality -> PII -> tokenize 串成端到端小流水。

一个最小可运行心智模型是：

```
mock or WARC docs
-> extract text
-> MinHash dedup
-> heuristic quality filter
-> PII/toxicity cleanup
-> SentencePiece tokenizer
-> report stage counts
```

这和 Pile 的真实系统相比很小，但学习价值在于：你能看到每个阶段如何改变样本数、文本分布和最终训练材料。

16. 代码样例：把 Pile 的权重思想写成玩具采样器

论文中的 epoch 权重可以用一个非常小的采样器表达：

```

import random

class MixSampler:
    def __init__(self, domains, epochs, seed=42):
        total = sum(len(domains[k]) * epochs[k] for k in domains)
        self.keys = list(domains)
        self.probs = [
            len(domains[k]) * epochs[k] / total
            for k in self.keys
        ]
        self.domains = domains
        self.rng = random.Random(seed)

    def sample(self, n):
        for _ in range(n):
            domain = self.rng.choices(self.keys, self.probs)[0]
            yield domain, self.rng.choice(self.domains[domain])

```

这里不要只看代码，要问三个实验问题：

1. 如果把 code 从 1 epoch 提到 3 epochs，代码样本比例如何变？
2. 如果一个小数据集被上采样太多，重复 exposure 是否会导致记忆？
3. 如果质量过滤先删掉大量非网页文本，再调权重，还有没有办法恢复多样性？

这就是从“会跑脚本”进入“能做数据实验”的分水岭。

17. 代码样例: BPB 计算

BPB 的计算可以写成下面这样。实际模型 loss 会由 PyTorch/HF 模型给出，这里只保留公式结构：

```

import math

def bits_per_byte(nll_loss, n_tokens, n_utf8_bytes):
    token_per_byte = n_tokens / max(n_utf8_bytes, 1)
    return token_per_byte * nll_loss / math.log(2)

text = "def f(x): return x * x"
n_bytes = len(text.encode("utf-8"))
n_tokens = 8
nll = 1.25
print(bits_per_byte(nll, n_tokens, n_bytes))

```

这个小函数能帮你理解论文为什么报告 BPB：同一段代码如果 tokenizer A 切成 8 个 token，tokenizer B 切成 12 个 token，token-level loss 的含义会变；BPB 把比较单位拉回 byte。

18. 代码样例: 13-gram contamination 检测

论文用 13-gram overlap 处理训练/评测重叠。教学版可以这样写：

```
def word_ngrams(text, n=13):
    words = text.lower().split()
    return {
        tuple(words[i:i + n])
        for i in range(len(words) - n + 1)
    }

def overlaps_eval(train_doc, eval_ngrams, n=13):
    return bool(word_ngrams(train_doc, n) & eval_ngrams)
```

这个检测只能抓直接重叠，抓不到改写、翻译、摘要式污染。但它足够便宜、可解释，是大规模数据进入评测前必须做的第一层卫生检查。

19. 对现在的意义

今天看 The Pile，它已经不是“最先进的数据配方”。后来的 FineWeb、DCLM、Dolma、RedPajama、Llama 3 数据工程在去重、质量分类器、教育性打分、多语言、benchmark contamination、metadata 审计上都更系统。

但 The Pile 的历史意义仍然很大：

- 它让开放社区有了可训练 GPT-Neo/GPT-J/GPT-NeoX 级模型的大数据基础。
- 它把数据配方和处理代码公开，降低了复现门槛。
- 它证明多域 mixture 能改善跨域建模，不只是“网页越多越好”。
- 它把数据文档化、安全、同意和版权问题放进技术论文正文。
- 它让后来的数据集论文必须回答更细的问题：你从哪里来？怎么过滤？怎么去重？怎么防污染？怎么评估质量？

学习时不要把 Pile 当成终点，而要把它当成起点：从它开始，你能读懂后来所有 data-centric LLM 论文在修补什么。

20. 局限和批判性阅读

The Pile 的局限必须明确写在笔记里：

1. 没有全局去重。跨组件重复和 train/test 残留重复都可能存在。
2. 英文为主。作者估计约 97.4% 英文，不适合作为多语言语料代表。
3. 质量过滤早期。Pile-CC 仍依赖 OpenWebText2 风格高质量代理，今天看不够精细。
4. 部分数据来源存在版权、同意和 ToS 争议，作者自己也花很多篇幅讨论。
5. Bias 分析是初步共现分析，不能证明模型训练后的真实行为风险。
6. 实验规模有限。1.3B / 40GB 控制实验能证明数据多样性有益，但不能给出今天千亿 token 级训练的最优配比。
7. Benchmark contamination 处理不彻底。13-gram 只能处理精确重叠，不能处理语义污染。

批判它不是否定它。真正的学习姿态是：理解它在当时解决了什么，再理解后来的工作为什么继续改。

21. 用 AI agent 正确学习这篇论文

这篇论文特别适合训练你和 agent 协作学习，因为它既有论文叙事，也有本地代码和可跑实验。建议采用三轮法。

第一轮：让 agent 考你结构，而不是替你总结

提示词：

我正在读 The Pile。请你只根据我给出的笔记来提问，不要直接替我总结。一次只问一个问题。

问题顺序按：

背景痛点 -> 22 个子集 -> mixture weights -> BPB ->

实验设计 -> 去重/污染 -> 局限。

我答完后，请指出我漏掉的证据链。

第二轮: 让 agent 把论文 claim 映射到证据

你可以要求 agent 建一张 claim/evidence 表：

请把 The Pile 的主要 claim 拆成：

claim, paper evidence, possible confounder,

local mini-experiment, related source file。

不要写泛泛解释，每条 claim 必须能落到一个表格或段落。

你自己必须检查它有没有把 Table 3 / Table 4 / D.2 / 6.x 节混在一起。Agent 很容易把 “数据质量更好” 说得太泛，你要逼它指出：到底是哪项 BPB、哪个训练对比、哪个组件改善。

第三轮: 让 agent 陪你跑小实验，但实验解释必须你写

建议实验：

- 跑 minhash_dedup.py --demo，解释 threshold 变化对 removed 数的影响。
- 跑 data_mix_ablation.py --demo，解释不同 domain weights 如何改变 mock ppl。
- 跑 capstone_mini_corpus.py --smoke，记录每个阶段样本数如何变化。

你要把结果写成五句话：

1. 我改了哪个变量。
2. 它对应论文哪个机制。
3. 指标如何变化。
4. 变化是否符合论文直觉。
5. 这个 toy 实验不能证明什么。

这五句话才是知识进入脑子的过程。只让 agent 生成一篇摘要，不会让你掌握数据工程。

22. 阅读后的闭卷问题

读完后，你应该能闭卷回答：

1. The Pile 为什么不是 “一个更大的 Common Crawl”？
2. 22 个子集大致分成哪几类？每类补充模型什么能力？
3. Weight、Epochs、Effective Size 三列分别表达什么？
4. 为什么高质量小数据集要上采样？上采样有什么风险？
5. BPB 的公式是什么？为什么它比普通 perplexity 更适合 Pile？
6. GPT-3 component-wise 分析为什么要用 OpenWebText2 做参照？
7. Table 3 说明 Pile 相对 CC-100 和 Raw CC 的优势在哪里？
8. 为什么 CC-100 的 Wikipedia-perplexity filtering 可能损害多样性？
9. Pile 在去重上做了什么，没有做什么？
10. 13-gram overlap filtering 能防什么，不能防什么？
11. Pile 的数据文档化包括哪些维度？
12. 如果你要在本仓库复现一个最小实验，会改哪个文件，看哪个指标？

23. 一页复盘

The Pile 的 story 是：大模型需要更多、更广、更透明的数据；Common Crawl 虽大但噪声和分布单一问题明显；作者把 22 个高价值来源混成 825.18 GiB 英文训练集，并通过 epoch 权重上采样专业/高质量子集。它把训练集同时设计成 benchmark，用 BPB 评估跨 tokenizer 的语言建模质量。实验中，同样 40GB 训练预算下，Pile 训练的 1.3B 模型在 Pile 全部子集上优于 CC-100 和 Raw CC，在 WikiText 上也更好。论文还详细讨论去重、13-gram 污染、主题分布、语言比例、毒性、偏见、版权和同意。它的局限是没有全局去重、英文为主、过滤和污染处理不够现代、数据来源存在争议。它的长期意义是把开放 LLM 的数据配方变成了可复现、可审计、可批评的工程对象。

24. 本仓库下一步学习动作

按这个顺序做：

1. 读 `learning/data-curation/lectures/01-data-overview.md`，先建立 C4 -> Pile -> FineWeb -> DCLM 的时间线。
2. 跑数据配比玩具实验：`learning/data-curation/src/data_mix_ablation.py --demo` 观察不同 mixture 权重。
3. 跑去重玩具实验：`learning/data-curation/src/minhash_dedup.py --demo` 理解近重复召回。
4. 跑端到端 smoke pipeline：`learning/data-curation/src/capstone_mini_corpus.py --smoke` 看 `extract/dedup/quality/pii/tokenize` 每阶段怎样改变 corpus。
5. 写一页自己的“数据配方审计表”：`source`、`license/consent`、`filter`、`dedup`、`contamination`、`eval metric`、`known risk`。

完成这五步，你读到的就不只是 The Pile 这篇论文，而是现代 LLM 数据工程的起点。